

Practical 07 - Linux Process Memory Analysis

Objective

This practical studies the Linux process address space and virtual memory mappings.

Part 1

Write a C program that prints the addresses of:

- Code
- Global variables
- Static variables
- Heap
- Stack

Part 2

Use `/proc/<PID>/maps` and memory-analysis tools such as `pmap` to study the virtual memory organization of a running process.

Files

- `memory_layout.c` - Displays addresses of different memory segments.
- `memory_maps.c` - Reads and displays `/proc/<PID>/maps`.
- `README.md` - Practical documentation.
- `output.txt` - Sample output.

Part 1 - Memory Layout

Compile:

```
gcc -Wall -Wextra -g memory_layout.c -o memory_layout
```

Run: `./memory_layout`

The program displays the addresses of:

Code Segment

Contains the machine instructions of the program.

A `function address` is used to demonstrate the code segment.

Global Variables

Global variables have program-wide scope and are stored `in` the program's `data-related` memory area.

Static Variables

Static variables have static storage duration and are stored in the program's `data-related` memory area.

Heap

The heap is used `for` dynamic memory allocation.

The program uses: `malloc()`
to allocate heap memory and:

`free()` to release it.

Stack

The stack stores `local` variables and `function call` information.

A `local` variable inside `main()` is used to demonstrate the stack.

Part 2 - `/proc/PID/maps`

Linux exposes information about a running process through the `/proc` virtual filesystem.

The file: `/proc/<PID>/maps`

contains the virtual memory mappings of the process.

For example: `cat /proc/<PID>/maps`

The mapping format is: START-END PERMISSIONS OFFSET DEVICE INODE PATH

Permissions

r - Read

w - Write

x - Execute

p - Private

s - Shared

Important Mappings

Program Code

The executable code is normally mapped with `read` and execute permissions:

`r-xp`

Data

Writable program data is normally mapped with:

`rw-p`

Heap

The mapping marked:

`[heap]`

represents dynamically allocated process memory.

Stack

The mapping marked:

`[stack]`

represents the process stack.

Shared Libraries

Libraries such as `libc.so.6` are mapped into the process's virtual address space.

VDSO

The [vdso] mapping is a kernel-provided virtual shared object that allows certain operations to be performed efficiently from user space.

pmap Analysis

The pmap command can be used to inspect a process's memory mappings.

pmap <PID>

For extended information:

pmap -x <PID>

This provides information such as:

Virtual memory size

Resident memory

Dirty memory

Permissions

Mapping names

Linux Process Address Space

A simplified process address space can be represented as:

Higher Addresses



Lower Addresses

The exact layout and addresses depend on the architecture, operating system configuration, executable type, and security mechanisms such as ASLR.

Virtual Memory

Each Linux process normally operates **in** its own virtual address space.

Virtual addresses are translated to physical memory by the operating system and hardware memory-management mechanisms.

Different processes can therefore use similar virtual address ranges **while** referring to different physical memory.

Memory mappings also provide protection by controlling whether regions can be read, written, or executed.

Conclusion

The practical demonstrates the organization of a Linux process's virtual address space.

The addresses of code, global/static variables, heap, and stack were observed using a C program.

The `/proc/<PID>/maps` file and `pmap` were used to inspect virtual memory mappings of a running process.

This provides practical understanding of Linux process memory organization and virtual memory.

Memory_layout.c -

```
#include <stdio.h> #include <stdlib.h> #include <unistd.h>
```

```
/* Global variable */ int global_variable = 100;
```

```
/* Static global variable */ static int static_global_variable = 200;
```

```
/* Function - code segment */ void sample_function() { printf("Inside  
sample_function()\n"); }
```

```
int main() { /* Local variable - stack */ int stack_variable = 300;
```

```
/* Static local variable - data segment */  
static int static_local_variable = 400;
```

```
/* Heap variable */  
int *heap_variable = malloc(sizeof(int));
```

```

if (heap_variable == NULL)
{
    perror("malloc");
    return 1;
}

*heap_variable = 500;

printf("=====\n");
printf("      LINUX PROCESS MEMORY LAYOUT\n");
printf("=====\n");

printf("Process PID           : %d\n", getpid());

printf("\nMemory Addresses:\n");
printf("-----\n");

printf("Code (function)          : %p\n",
      (void *)sample_function);

printf("Global variable          : %p\n",
      (void *)&global_variable);

printf("Static global variable   : %p\n",
      (void *)&static_global_variable);

printf("Static local variable    : %p\n",
      (void *)&static_local_variable);

printf("Heap variable            : %p\n",
      (void *)heap_variable);

printf("Stack variable           : %p\n",
      (void *)&stack_variable);

printf("-----\n");

printf("\nMemory Segment Analysis:\n");
printf("Code   -> Contains program instructions.\n");
printf("Global -> Stores global variables.\n");
printf("Static -> Stores static variables.\n");
printf("Heap   -> Dynamically allocated memory.\n");

```

```

printf("Stack -> Stores local variables and function data.\n");

printf("\nHeap value : %d\n", *heap_variable);
printf("Stack value : %d\n", stack_variable);

printf("\nProcess is running...\n");
printf("PID: %d\n", getpid());
printf("Open another terminal and run:\n");
printf("cat /proc/%d/maps\n", getpid());
printf("pmap %d\n", getpid());

printf("\nPress Enter to terminate...\n");
getchar();

free(heap_variable);

printf("Heap memory released successfully.\n");

return 0;

}

```

Memory_maps.c-

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

```

```

int main()

```

```

{

```

```

    char path[64];

```

```

    char line[512];

```

```

    printf("=====\n");

```

```

    printf("    /proc/PID/maps MEMORY ANALYSIS\n");

```

```

    printf("=====\n");

```

```
printf("Process PID: %d\n", getpid());
```

```
snprintf(path, sizeof(path),  
         "/proc/%d/maps", getpid());
```

```
FILE *fp = fopen(path, "r");
```

```
if (fp == NULL)
```

```
{  
    perror("fopen");  
    return 1;  
}
```

```
printf("\nVirtual Memory Mappings:\n");
```

```
printf("-----\n");
```

```
while (fgets(line, sizeof(line), fp) != NULL)
```

```
{  
    printf("%s", line);  
}
```

```
printf("-----\n");
```

```
fclose(fp);
```

```
printf("\nAnalysis:\n");
```

```
printf("[heap] -> Dynamic memory allocation area.\n");
```

```
printf("[stack] -> Process stack area.\n");  
printf("r-xp  -> Read and execute permissions.\n");  
printf("rw-p   -> Read and write permissions.\n");  
printf("Shared libraries are mapped into the process address space.\n");  
  
return 0;  
}
```

Output.txt-

```
=====
```

PRACTICAL 07 OUTPUT

```
=====
```

PART 1: PROCESS MEMORY LAYOUT

Compilation:

```
gcc -Wall -Wextra -g memory_layout.c -o memory_layout
```

Execution:

```
./memory_layout
```

Sample Output:

```
=====
```

LINUX PROCESS MEMORY LAYOUT

=====

Process PID : 12345

Memory Addresses:

Code (function) : 0x55XXXXXXXX

Global variable : 0x55XXXXXXXX

Static global variable : 0x55XXXXXXXX

Static local variable : 0x55XXXXXXXX

Heap variable : 0x55XXXXXXXX

Stack variable : 0x7fXXXXXXXX

Memory Segment Analysis:

Code -> Contains program instructions.

Global -> Stores global variables.

Static -> Stores static variables.

Heap -> Dynamically allocated memory.

Stack -> Stores local variables and function data.

Heap value : 500

Stack value : 300

PART 2: /proc/PID/maps

Command used:

```
cat /proc/12345/maps
```

The output showed mappings for:

- Executable program
- Code/text segment
- Data segment
- Heap
- Shared libraries
- Stack
- VDSO
- VVAR

Important examples:

r-xp -> Read and execute

rw-p -> Read and write

[heap] -> Heap memory

[stack] -> Stack memory

[vdso] -> Virtual Dynamic Shared Object

PART 3: PMAP ANALYSIS

Command:

```
pmap 12345
```

Extended command:

```
pmap -x 12345
```

The output displayed virtual memory regions and memory usage.

Observed mappings included:

- Program executable
- Shared libraries
- Heap
- Stack
- Anonymous mappings

PART 4: MEMORY MAP PROGRAM

Compilation:

```
gcc -Wall -Wextra -g memory_maps.c -o memory_maps
```

Execution:

```
./memory_maps
```

The program successfully opened:

/proc/<PID>/maps

and displayed the virtual memory mappings of the running process.

=====

PRACTICAL COMPLETED

=====